LECTURE 07 · AI 보조 개발

01 / 10

코드에서 대화로:
미국 시장 뉴스 보드 만들기

전통적 코딩, 프롬프트 코딩, 에이전트 코딩의 차이를 **경제 뉴스 보드(Econ News Board)** 구축 예시로 배웁니다.

① 전통적 코딩 | ② 프롬프트 코딩 | ③ 에이전트 코딩

정재식 교수 · 서강대학교 × WWAI | 2025년 봄학기

AGENDA

02 / 10

오늘 수업 로드맵

01 **프로젝트 소개 — 경제 뉴스 보드**
만들 것: 실시간 데이터 + 뉴스 헤드라인이 있는 미국 금융 시장 대시보드

02 **전통적 코딩 방식**
순수 Python + HTML/CSS/JS: 직접 설계하고, 구현하고, 디버깅. 완전한 이해, 완전한 노력.

03 **프롬프트 코딩 (바이브 코딩)**
원하는 것을 Claude / GPT-4에게 설명하고, 자연어로 반복 개선. 훨씬 빠른 개발.

04 **에이전트 코딩**
LLM + 도구: 검색 → 수집 → 분석 → 작성 → 배포. 자율적 다단계 파이프라인.

05 **비교, 데모 & 실습**
언제 어떤 방식을? 속도·비용·신뢰성·투명성 측면의 트레이드오프.

THE PROJECT 03 / 10

🗞 경제 뉴스 보드 — 우리가 만들 것

목표 출력물

▸ 미국 시장 데일리 2025.03.11 · 09:32 EST

SPY 512.40 ▲ +1.23%	QQQ 431.20 ▼ −0.44%	DXY 103.81 ▲ +0.18%
GLD 188.70 ▲ +0.72%	BTC 62,310 ▼ −2.11%	VIX 18.42 ▲ +3.20%

연준	파월, 인플레이션 개선 전까지 금리 인하 없다고 시사
WSJ	기술주 실적 예상치 상회, 나스닥 15,000선 회복
BBG	관세 우려로 소비재 섹터 하락, 무역 긴장 고조

기능 요구사항

📈 시장 데이터 레이어
SPY, QQQ, DXY, GLD, BTC, VIX 실시간 가격 (Alpha Vantage / Yahoo Finance API)

📰 뉴스 수집
연준, WSJ, Bloomberg, Reuters의 상위 5개 헤드라인 — 금융 키워드 필터링

🎨 프론트엔드 화면
반응형 HTML 대시보드, 상승/하락 색상 구분, 60초마다 자동 갱신

☁️ 배포
GitHub Pages 또는 Streamlit Cloud에 배포 가능한 단일 페이지 앱

방식 01 04 / 10

🔨 전통적 코딩

개발자가 모든 코드를 직접 작성. 완전한 이해, 완전한 노력.

PYTHON 백엔드 — DATA_FETCH.PY

```python
# data_fetch.py  — 전통적 방식
import requests, json
from datetime import datetime

TICKERS = ["SPY", "QQQ", "GLD", "BTC-USD"]
API_KEY = "YOUR_ALPHA_VANTAGE_KEY"

def fetch_price(symbol):
    url = (
      f"https://www.alphavantage.co/query?"
      f"function=GLOBAL_QUOTE&symbol={symbol}"
      f"&apikey={API_KEY}"
    )
    r = requests.get(url)
    data = r.json()
    return {
      "price": data["Global Quote"]["05. price"],
      "change": data["Global Quote"]["10. change percent"],
    }

def build_dashboard():
    results = {}
```

개발 과정

① 아키텍처 설계 (~2시간)

데이터 흐름 계획: API → Python → JSON → HTML. 라이브러리 선택, API 문서 읽기.

② 모든 모듈 작성 (~6시간)

data_fetch.py, news_scraper.py, render_html.py, style.css — 모두 처음부터 작성.

③ 디버깅 & 테스트 (~4시간)

API 요청 제한, JSON 키 오류, CORS 문제, CSS 반응형 버그 등.

④ 배포 (~1시간)

GitHub Actions 설정, Pages 배포, 환경변수 등록.

⏱ 예상 총 소요 시간: 12~16시간

Python, REST API, HTML/CSS, 배포 파이프라인 전반에 걸친 숙련도 필요

방식 02 05 / 10

💬 프롬프트 코딩 (바이브 코딩)

의도를 자연어로 설명 → LLM이 코드 작성 → 사람이 검토하고 반복 개선

프롬프트 시퀀스 (CHATGPT / CLAUDE)

— 프롬프트 1 —

```
"Alpha Vantage API를 사용해서 SPY, QQQ,
GLD, BTC-USD의 실시간 가격을 가져와
JSON으로 반환하는 Python 스크립트를
작성해줘."
```

— 프롬프트 2 (개선) —

```
"이 JSON 데이터를 Bloomberg 스타일 카드
그리드로 보여주는 HTML 페이지를 만들어줘.
상승은 초록, 하락은 빨강으로 표시해."
```

— 프롬프트 3 (뉴스 추가) —

```
"하단에 NewsAPI에서 미국 금융 뉴스를
키워드 필터링해서 가져오는 섹션 추가해줘."
```

— 프롬프트 4 (자동 갱신) —

```
"페이지 전체 새로고침 없이 60초마다
데이터를 자동 갱신하게 해줘. fetch() API
사용해서."
```

핵심 특징

✓ 장점

- 초기 구조 작성 10배 빠름
- 문법 암기 불필요
- 즉시 동작하는 보일러플레이트
- 대화로 쉽게 반복 개선

✗ 한계

- 프롬프트 품질에 결과 달라짐
- '블랙박스' 코드 디버깅 어려움
- API 엔드포인트 환각 가능
- 컨텍스트 창 한계로 복잡도 제한

바이브 코딩의 "바이브"란?

원하는 *느낌*과 *동작*을 설명하면, 모델이 구현 세부사항을 채워줍니다. 개발자의 역할이 **직접 쓰기**에서 **방향 제시**로 바뀝니다.

⏱ 예상 소요 시간: **1~3시간** (대부분 프롬프트 반복 + 검토)

방식 03 06 / 10

🤖 에이전트 코딩

LLM + 도구 + 메모리: 자율적 다단계 실행. 목표만 주면 에이전트가 알아서 처리.

에이전트 실행 흐름 (CLAUDE CODE / AUTOGEN / LANGGRAPH)

🎯 목표: "뉴스 보드 만들기" → 🔍 web_search "Alpha Vantage 문서" → 🌐 web_fetch API 엔드포인트 수집 → 🧠 아키텍처 설계 → 📝 write_file data_fetch.py → ⚡ run_bash python test.py → 🔄 오류 수정 (자동 루프) → 🚀 배포 git push

Claude Code 예시

```
> claude-code

"Alpha Vantage + NewsAPI로
미국 금융 뉴스 보드 앱을
만들고 GitHub Pages에
배포해줘."

# 에이전트 자동 실행:
# 1. API 문서 검색
# 2. 모든 파일 작성
# 3. 테스트 & 버그 수정
# 4. 커밋 & 배포
```

에이전트가 할 수 있는 것

- 🔍 최신 API 문서 웹 검색
- 📂 로컬 파일 읽기/쓰기
- ⚡ 터미널 명령 실행
- 🐛 오류 읽기 → 수정 → 재실행
- 🔀 복수의 서브에이전트 조율
- 📊 출력 품질 자체 검증

주의 사항

- 💸 API 비용 (LLM 호출 많음)
- 🎲 비결정적 동작 가능
- 🔒 도구 권한 관리 필요
- 🧪 모든 의사결정 감사 어려움
- ⚙️ 오케스트레이션 설정 필요

⏱ 예상 소요 시간: **15~45분** (사람은 결과 검토만, 에이전트가 작업)

비교 07 / 10

세 가지 방식 나란히 비교

비교 항목	① 전통적 코딩	② 프롬프트 코딩	③ 에이전트 코딩
코드 작성 주체	개발자 100%	LLM ~80%, 사람 20%	에이전트 ~95%, 사람 검토
MVP 완성 시간	12~16시간	1~3시간	15~45분
필요 역량	풀스택 개발 능력	기초 프로그래밍 + 프롬프팅	프롬프트 엔지니어링 + 오케스트레이션
디버깅	수동, 정밀	프롬프트로 반수동	자동 루프, 불투명
결과 일관성	결정적 ✓	좋은 프롬프트로 높음	가변적, 검증 필요
API 비용	없음 (시간만)	저렴 ($0.01~0.10)	중간 ($0.50~5.00)
적합한 상황	프로덕션, 학습	프로토타입, 과제	복잡한 파이프라인, 자동화
WWAI 적용 예시	LRS 시그널 엔진 코어	대시보드 UI 프로토타이핑	GNN 데이터 파이프라인 자동화

라이브 데모 08 / 10

🖥 데모: 같은 보드를 3가지 방식으로 만들기

① 전통적 (사전 제작)

```python
# news_scraper.py
import feedparser

FEEDS = {
  "로이터": "https://feeds.reuters.com/reuters/businessNews",
  "블룸버그": "https://feeds.bloomberg.com/markets/news.rss",
}

def get_headlines(n=5):
    items = []
    for src, url in FEEDS.items():
        feed = feedparser.parse(url)
```

총 300줄 이상, 파일 4개

② 프롬프트 코딩 (실습)

```
나: "미국 시장 뉴스 보드를 보여주는
단일 HTML 파일을 만들어줘.
티커: SPY, QQQ, GLD, BTC, VIX.
지금은 목업 데이터로 하되, 나중에
실제 API로 교체할 수 있는 구조로.
블룸버그 스타일 다크 테마."

── Claude가 ~150줄의 완성된 ──
── HTML/JS/CSS 코드를 제공 ──

나: "뉴스 섹션에 타임스탬프 추가하고,
가격 갱신 시 잠깐 깜빡이게 해줘."
```

프롬프트 ~4개 · 20분 · 파일 1개

③ 에이전트 (CLAUDE CODE)

```
$ claude-code

> "미국 금융 뉴스 보드 앱 만들어줘.
요구사항:
- Alpha Vantage로 가격 데이터
- NewsAPI로 헤드라인
- 60초 자동 갱신
- GitHub Pages에 배포
- README 포함"

에이전트: Alpha Vantage 문서 검색 중…
에이전트: NewsAPI 레퍼런스 읽는 중…
```

명령 1개 · 12분 · 완전 자율

💡 핵심 인사이트

같은 결과물, 완전히 다른 과정. 어떤 방식을 선택할지는 학습 목표, 시간 제약, 감사 필요성, 시스템 복잡도에 따라 달라집니다.

요약 09 / 10

성적표

① 전통적 코딩

속도	★★
학습 효과	★★★★★
신뢰성	★★★★★
확장성	★★★
접근성	★★

추천 상황: 프로덕션 시스템, 기초 학습, 규제 환경

② 프롬프트 코딩

속도	★★★★
학습 효과	★★★
신뢰성	★★★
확장성	★★★
접근성	★★★★★

추천 상황: 빠른 프로토타입, 학술 과제, 새 라이브러리 탐색

③ 에이전트 코딩

속도	★★★★★
학습 효과	★★
신뢰성	★★★
확장성	★★★★★
접근성	★★★

추천 상황: 복잡한 파이프라인, 자동화, 검증 가능할 때

"세 가지 방식을 모두 활용할 수 있는 개발자는
3배가 아니라 ***10배*** 더 생산적입니다."

— 이 수업의 목표

실습 10 / 10

🏋️ 수업 내 실습

과제 (2인 1조)

Task A — 전통적 코딩 (30분)

requests 라이브러리와 Alpha Vantage API를 사용해 `get_spy_price()` 함수를 AI 도움 없이 처음부터 작성하세요. 모든 줄에 주석을 달아야 합니다.

Task B — 프롬프트 코딩 (20분)

Claude.ai를 사용해 ≤5개의 프롬프트로 전체 HTML 뉴스 보드를 만드세요. 사용한 모든 프롬프트를 기록하고, 무엇이 잘 됐고 무엇이 실패했는지 분석하세요.

Task C — 에이전트 코딩 (10분)

Claude Code에 단 하나의 지시로 보드를 만들고 저장하세요. 에이전트가 몇 단계를 수행했는지 세어보고, git 로그를 확인하세요.

성찰 질문

Q1. 이해의 깊이

Task A vs B/C 이후: 어느 버전의 코드를 실제로 이해하고 있나요? 프로덕션 장애 시 디버깅할 수 있을까요?

Q2. 프롬프트 품질

프롬프트 표현이 어떻게 결과 품질을 바꿨나요? 가장 나빴던 프롬프트와 그 결과물은 무엇이었나요?

Q3. 신뢰 조정

Task C에서 에이전트가 당신이라면 하지 않았을 결정을 내린 적이 있나요? API 오류를 올바르게 처리했나요?

📤 제출물: 세 가지 방식을 비교하는 A4 1페이지 PDF 제출. 포함 내용: 소요 시간, 코드 줄 수, 버그 수, 배운 점.